Toward a Capability Specification for Kubernetes-Based Supervision of Process Plants

Matheus Corteletti Delfino
Control and Automation Engineering
Instituto Federal do Espírito Santo (Ifes)
Serra, ES, Brazil
matheus.corteletti.del@gmail.com

Rafael Emerick Zape de Oliveira
Control and Automation Engineering
Instituto Federal do Espírito Santo (Ifes)
Serra, ES, Brazil
rafael.emerick@ifes.edu.br

***Abstract*—Kubernetes keeps software systems close to a declared desired state, and recent work uses it to run industrial control software. This work in progress asks a different question: what must an implementation demonstrate before one can say that a process plant is under Kubernetes-based supervision? We propose a specification organized in four verifiable capabilities: observing the operation against a declared policy, evaluating and executing operating changes, managing devices and versions, and integrating equipment through OPC UA. We then report a first open implementation that demonstrates the first capability on a simulation of the Tennessee Eastman Process. The simulated plant reproduces the published operating costs of the base case (170.63 against 170.61 $/h) and of the Mode 1 optimum (114.32 against 114.31 $/h). With the plant running about ten times faster than real time, a Kubernetes controller evaluated it against the declared policy every 30 s and flagged a feed loss 1.15 h of process time after it occurred. In another run, cost fell 37% in 30 h while the plant drifted toward shutdown, and only a declared level limit flagged it, 2 h before the shutdown. We describe the plan for the remaining capabilities and a validation protocol open to other implementations.**

***Index Terms*—Kubernetes, process supervision, operating policy, OPC UA, Tennessee Eastman Process, digital twin, reference architecture**

I. Introduction

Process plants are operated in layers [1]. Controllers keep variables such as levels and pressures at their setpoints, and a supervisory layer chooses those setpoints and checks whether the plant does what was decided. In the enterprise-control hierarchy of IEC 62264 [2], this supervision sits between the control systems and the business systems. In practice, the engineers who know how to improve a plant (chemical, mechanical and control engineers) often work on data exported from the historian and analyzed offline.

Software engineering faces a similar problem in distributed systems. Kubernetes is a container orchestration platform with a declarative interface: the user declares the desired state as resources, and controllers continuously bring the observed state toward it [3]. New resource types and controllers can be added without changing the platform. This suggests using Kubernetes as the supervisory layer of a plant: the operating policy becomes a declared resource, and a controller reconciles the plant against it.

Using Kubernetes is not enough to say that a plant is supervised. A company that claims to follow an information security standard must show that certain processes exist, regardless of the tools it uses. In the same spirit, we ask what an implementation must demonstrate before one can say that a plant is under Kubernetes-based supervision. This paper contributes:

- a specification organized in four verifiable capabilities, each with what an implementation must demonstrate (Section III);
- a first open implementation that demonstrates the first capability on the Tennessee Eastman Process (TEP) [4], with results (Sections IV and V);
- a plan for the remaining capabilities and a validation protocol open to other implementations (Section VI).

The specification is a proposal, not a standard. We submit it at this stage to receive criticism before the remaining capabilities are built.

II. Related Work

Kubernetes has reached industrial automation mainly as infrastructure for control software. Johansson et al. use it to recover failed controllers with high availability [5], Mellado and Núñez run a programmable logic controller in containers [6], and Koziolek and Burger use a purpose-built Kubernetes operator to update virtual controllers without halting production [7]. In these cases, Kubernetes manages the software that controls the process, not the decisions about how the process should operate. Cloud-native practice in general is surveyed in [8].

The supervisory functions themselves have long roots. Autonomic computing organizes self-management as monitoring, analysis, planning and execution over shared knowledge [9]. Reference governors change setpoints so that constraints are respected during transitions [10], [11], and digital twins support prediction and control [12], [13]. OPC UA is the standard way for equipment to expose data and methods [14]. The works above do not define what a Kubernetes-based supervisory layer must demonstrate, which is the gap this paper addresses.

III. Capability Specification

Table I lists the four capabilities. Kubernetes is the base that coordinates them; the specification gives them an industrial

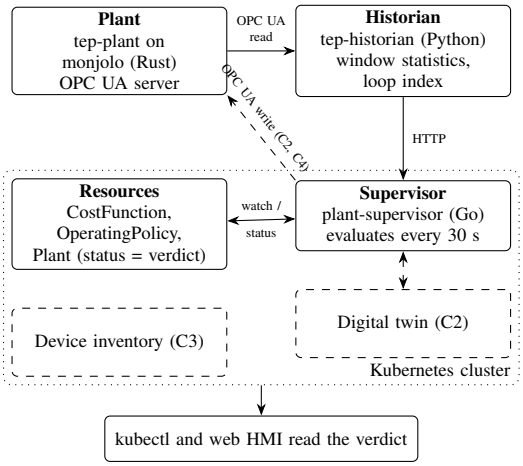


Fig. 1. Architecture of the first implementation. Solid: built and evaluated. Dashed: planned.

meaning and states the conditions for considering them met. An implementation may demonstrate only some capabilities, and it reports which ones and with what evidence.

Three design rules apply to all capabilities. First, the supervisor knows the plant only through the declared resources, so that the same supervisor can, in principle, serve another plant; this has not been tested yet. Second, verdicts and decisions are stored in Kubernetes itself, where any client can read them and where they leave a record. Third, control and safety functions stay in the plant controllers: the supervisor is never in the real-time path, and the plant must stay safe if the supervisor fails.

IV. First Implementation

The implementation is open source[1] and has four components (Fig. 1).

Plant. The Downs and Vogel model [4] was rewritten in Rust as a plant that runs continuously and publishes its signals over OPC UA. The generic part (state registry, integrator, actuator and sensor blocks, OPC UA adapter) was separated into a runtime called `monjolo`, which knows nothing about the TEP. The plant can run faster than real time.

Historian. A Python service collects every signal over OPC UA, keeps the last 15 minutes, and serves window statistics and a control loop performance index [15] over HTTP. Kubernetes never sees raw signals, only these aggregates.

Supervisor. A Kubernetes controller written in Go reads three custom resources. `CostFunction` declares the operating cost as a sum of terms; `OperatingPolicy` declares the cost budget, production targets, variable limits, control loops to watch, and how many bad evaluations in a row change the verdict; `Plant` links them to a plant and holds the verdict. Every 30 s, the supervisor queries the historian and writes into the `Plant` status the conditions `CostWithinBudget`, `ConstraintsSatisfied` and `ControlLoopsHealthy` and a phase, compliant or not. The policy can be edited with `kubectl` while the supervisor runs, and is re-evaluated immediately.

Interface. The verdict is read with `kubectl` or in a web HMI. The cluster is a single local node.

[1] https://github.com/Green-Cinnamon-Labs.
Start at the `spec-tennessee-eastman` repository.

V. Preliminary Results

All times below are process times. The plant was accelerated, but the 30 s evaluation period is wall-clock time, so each evaluation covered more process time than it would in real operation.

Model check. Table II compares the operating cost of the simulated plant with the published values. The Mode 1 value comes from a plant version with integrated cooling water. This checks the steady state of the model, not its dynamics.

Normal operation. The Mode 1 policy was calibrated on a run without disturbance, at about ten times real time: over 41 evaluations, the cost stayed at 166.83 ± 0.18 \$/h and all conditions held. The cost budget was set at 170.6 \$/h, the cost of the base case.

Feed loss. At about ten times real time (about 5 min of process per evaluation), with the policy fixed, disturbance IDV(6), the loss of feed A, was applied. The product flow left its target 0.98 h after the disturbance, and the cost exceeded the budget at 1.15 h (about 7 min of wall-clock time), when the phase changed to non-compliant: the verdict followed the physical order of events. The loop index alone did not flag the disturbance. A setpoint tracking criterion was added, which required changing the supervisor code, not only the policy; in a repeated run with it, the levels crossed the declared limit at 0.84 h and the loop condition turned false at 1.00 h. A later re-reading of the data showed that the index was computed on an inverted scale, which explains part of this blindness.

Cost that improves while the plant degrades. With the three control loops changed from proportional to PI, and no reactor level loop, the plant ran without disturbance at about 56 times real time (about 28 min of process per evaluation). The purge valve closed slowly, inert gas accumulated and the reactor level fell from 71.6% to 47.8% in 30 h, until the reactor pressure crossed the 3000 kPa shutdown limit at 32.9 h. Throughout this run, the cost fell, from 165.9 to 105.3 \$/h in 30 h, because less reactant left in the purge; the cost condition held in all 70 evaluations, and the loops stayed healthy. Only the reactor level limit declared in the policy (at least 50%) noticed, with a non-compliant verdict at 30.9 h, close to the physical loss. Part of this delay comes from the acceleration, but the level had also fallen for about 25 h inside the allowed range: a fixed limit warns when a variable arrives, not when it starts to drift.

What this shows for C1. A Kubernetes controller can follow a running simulated plant through a historian, from a policy that the user declares and edits. This was not tested in real time, on a physical plant or on a multi-node cluster. The runs also show the limits of observation: a useful policy must watch inventories and trends, not only cost and loops, and a new type of criterion still requires code. The last run is also the

TABLE I
CAPABILITIES OF KUBERNETES-BASED PLANT SUPERVISION

Capability	What an implementation must demonstrate	Status in this work
C1 Observe the operation	The supervisor receives enough up-to-date data to compare the plant with the declared policy, and records its verdict as Kubernetes state.	Demonstrated on the simulated TEP (Section V).
C2 Evaluate and execute changes	A change in the declaration starts a procedure: prepare a digital twin synchronized with the plant, test the change, evaluate the criteria, authorize or refuse, apply, and verify the plant response, including when the result is inconclusive or a component fails.	Planned. The current supervisor never writes to the plant.
C3 Manage devices and versions	Keep an inventory of devices, software and configuration versions and the history of changes, separating the declared state from the state found in the plant.	Planned.
C4 Integrate through OPC UA	Define how equipment exposes data and commands, with identification, permissions and handling of communication failures.	Reading demonstrated; writing, permissions and failures not yet evaluated.

TABLE II
OPERATING COST OF THE SIMULATED PLANT

Operating point	Published ($/h)	This work ($/h)
Base case [4]	170.61	170.63
Mode 1 optimum [16]	114.31	114.32

main motivation for C2: when the supervisor only observes, a correct verdict may come too late to act on.

VI. ONGOING WORK AND VALIDATION PLAN

C2: changes as progressive rollouts. We are designing the supervisor to treat a setpoint change as Kubernetes treats a software deployment: applied in steps, verified, and rolled back if a declared condition fails. Before each step, the supervisor synchronizes a digital twin with the plant and simulates the step faster than real time; if the simulation violates the policy, the step is refused and the refusal is recorded. Three strategies will be compared with the same code: applying the change at once, in steps, and in steps tested on the twin. The comparison will measure the area, duration and peak of limit violations, completed and refused changes, and decision latency, also with deliberate errors in the twin parameters.

Second plant. The target is a didactic level process at Ifes, where a PID in the plant controller drives a pump and the setpoint is exposed through an OPC server. Its model will first be written in `monjolo`; experiments on the physical plant depend on access and authorization, which have not been confirmed yet. In this plant, the supervisor will only change setpoints; control and protections stay in the controller.

C3 and C4. Devices and their software and configuration versions will be represented as Kubernetes resources, with the declared and the found state side by side. The OPC UA write path will be specified with identification, permissions and behavior under communication loss.

Validation by others. To let other groups build and check their own implementations, we plan to simplify the stack: a Python interface for plant models beside the Rust one, a historian template, and a single-command cluster setup. An implementation will claim the capabilities it meets and pass a set of scenario tests for each. We will also measure the effort to bring a new plant under supervision: code written, and changes needed in the supervisor, with the goal of none.

VII. OPEN QUESTIONS AND CONCLUSION

We would welcome reviewers' views on three questions. Is a capability specification the right way to define Kubernetes-based supervision, and are these four capabilities the right ones? Why Kubernetes rather than a supervisory sequence in SCADA or in the plant controller? Our answer is the life cycle of an operating change, which becomes declared, versioned, reconciled and recorded, using the same tools as other software systems; we would like to know whether this matters to practitioners. And what evidence would convince plant engineers that such a layer is useful?

In summary, the first capability has been demonstrated on a simulated benchmark, with its limits; the others are a research program, with criteria that any implementation can be checked against.

REFERENCES

[1] S. Skogestad, "Control structure design for complete chemical plants," *Computers & Chemical Engineering*, vol. 28, no. 1–2, pp. 219–234, 2004.

[2] International Electrotechnical Commission, "Enterprise-control system integration – Part 1: Models and terminology," IEC, Tech. Rep. IEC 62264-1:2013, 2013. [Online]. Available: https://webstore.iec.ch/en/publication/6675

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[4] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.

[5] B. Johansson, M. Rågberger, T. Nolte, and A. V. Papadopoulos, "Kubernetes orchestration of high availability distributed control systems," in *2022 IEEE International Conference on Industrial Technology (ICIT)*, 2022, pp. 1–8.

[6] J. Mellado and F. Núñez, "Design of an IoT-PLC: a containerized programmable logical controller for the Industry 4.0," *Journal of Industrial Information Integration*, vol. 25, p. 100250, 2022.

[7] H. Koziolek and A. Burger, "Dynamic updates of virtual PLCs deployed as Kubernetes microservices," in *Software Engineering 2022*, ser. Lecture Notes in Informatics, vol. P-320. Gesellschaft für Informatik, 2022.

[8] S. Deng, H. Zhao, B. Huang, C. Zhang, F. Chen, Y. Deng, J. Yin, S. Dustdar, and A. Y. Zomaya, "Cloud-native computing: a survey from the perspective of services," 2023. [Online]. Available: https://arxiv.org/abs/2306.14402

[9] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[10] E. G. Gilbert, I. Kolmanovsky, and K. T. Tan, "Discrete-time reference governors and the nonlinear control of systems with state and control constraints," *International Journal of Robust and Nonlinear Control*, vol. 5, no. 5, pp. 487–504, 1995.

[11] E. Garone, S. Di Cairano, and I. Kolmanovsky, "Reference and command governors for systems with constraints: A survey on theory and applications," *Automatica*, vol. 75, pp. 306–328, 2017.

[12] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.

[13] Y. Zhao, Y. Cai, and H. Jiang, "Digital twin-based subspace model predictive control for thermal power plant," *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 237, no. 7, pp. 1171–1186, 2023.

[14] International Electrotechnical Commission, "OPC unified architecture – Part 1: Overview and concepts," IEC, Tech. Rep. IEC 62541-1, 2025.

[15] B. Bradu, E. Blanco Viñuela, R. Marti, and F. Tilaro, "Automatic PID performance monitoring applied to LHC cryogenics," in *16th International Conference on Accelerator and Large Experimental Physics Control Systems (ICALEPCS 2017)*, 2018, p. WEAPL02.

[16] N. L. Ricker, "Optimal steady-state operation of the Tennessee Eastman challenge process," *Computers & Chemical Engineering*, vol. 19, no. 9, pp. 949–959, 1995.